

Análise de Desempenho com Funções Hash Criptográficas

Tecnologias de Registro Distribuído
Universidade Federal de Campina Grande

Filtros de Bloom são a solução clássica para checar a existência de elementos em um conjunto, usualmente utilizado para mitigar o custo de consulta em conjuntos com volume massivo de dados. Neste laboratório você vai **dimensionar corretamente o filtro** e, **avaliar empiricamente o desempenho** de diferentes funções hash criptográficas.

Atenção

Utilize como ponto de partida o código-base visto em aula ([hashing_bloom_filter.ipynb](#)).

Atividade 1: Dimensionamento dos filtros

Um Filtro de Bloom é caracterizado por três parâmetros principais:

- n : número de elementos que se deseja inserir no filtro.
- m : tamanho do array de bits (número de posições disponíveis). Quanto maior m em relação a n , menor a chance de colisões e, portanto, menor a taxa de falsos positivos.
- k : número de funções hash independentes aplicadas a cada elemento. Cada função hash indica uma posição do array a ser marcada na inserção, ou verificada na consulta. Existe um valor ótimo de k : poucos hashes deixam muitas posições vazias sem aproveitar bem o espaço; muitos hashes saturam o array rapidamente, aumentando os falsos positivos.

A relação entre esses parâmetros determina a **taxa de falsos positivos** p — a probabilidade de o filtro indicar que um elemento está presente quando, na verdade, ele nunca foi inserido.

O artigo de Bloom (1970) deriva as fórmulas que relacionam n , m , k e p de forma ótima. Você vai encontrá-las e utilizá-las na Atividade 1. Desse modo, após a leitura do artigo de Bloom, nesta atividade você vai determinar os parâmetros necessários para construir um Filtro de Bloom confiável em dois cenários.

Dica

Os artigos estão disponíveis em:

- BLOOM, Burton H. **Space/time trade-offs in hash coding with allowable errors.** *Communications of the ACM*, v. 13, n. 7, p. 422–426, 1970. <https://dl.acm.org/doi/10.1145/362686.362692>
- MITZENMACHER, Michael. **The Future of Internet Architecture.** *IEEE Internet Computing*, v. 9, n. 1, p. 78–81, 2005. <https://www.eecs.harvard.edu/~michaelm/postscripts/im2005b.pdf>

Parte 1.1: Extração das Fórmulas

Leia o artigo de Bloom (1970) e, em uma célula Markdown do seu notebook, responda:

- Quais são as fórmulas que permitem calcular m e k a partir de n e de uma taxa de falsos positivos desejada p ? Escreva-as usando a notação $\LaTeX$ com $$$\dots$$$ no Markdown do notebook.
 - Tamanho ótimo do array de bits (m)
 - Número ótimo de funções hash (k)
 - Taxa teórica de falsos positivos (p) em função de n , m e k
- Explique com suas palavras o que cada variável representa e por que existe um valor ótimo para k .

Atenção

As respostas desses itens (a,b) devem ser elaboradas **sem o uso de LLMs ou de qualquer ferramenta automática de geração de texto.**

Parte 1.2: Aplicando as fórmulas

Usando as fórmulas que você extraiu do artigo, calcule os parâmetros m e k para os dois cenários abaixo, adotando uma taxa de falsos positivos de 1% ($p = 0,01$):

Cenário	Descrição	Número de itens (n)
A	Catálogo de produtos distribuído	40 bilhões
B	Pipeline de dados global	5 trilhões

Para cada cenário, implemente os cálculos em Python e apresente em uma tabela:

- O valor de m em bits e sua conversão para **GB**.
- O valor de k arredondado para o inteiro mais próximo.
- A taxa de falsos positivos p recalculada com os valores arredondados de m e k — verifique se ainda está dentro do limite de 1%.

Dica

Assuma que é possível alocar arrays de bits contíguos na memória, o que é uma simplificação. Em cenários reais, a implementação do filtro pode precisar lidar com fragmentação de memória e outras limitações.

Em seguida, em uma célula Markdown, justifique: *esse dimensionamento é viável para um servidor com 8 GB de RAM? E com 128 GB?* Se não for viável em nenhum dos dois casos, que alternativa de engenharia permitiria distribuir o filtro?

Atenção

As respostas desses itens (a,b e c) devem ser elaboradas **sem o uso de LLMs ou de qualquer ferramenta automática de geração de texto.**

Questão Final

Em uma célula Markdown, responda:

Q1. O que acontece com p se você usar o valor de k arredondado para baixo versus para cima? Calcule os dois casos numericamente e discuta qual arredondamento é mais conservador do ponto de vista de confiabilidade do filtro.

Atenção

As respostas desse item deve ser elaborada **sem o uso de LLMs ou de qualquer ferramenta automática de geração de texto.**

Atividade 2: Análise de Desempenho

Nesta etapa você deve comparar o tempo de execução de três funções hash criptográficas usadas em um Filtro de Bloom, avaliando o impacto do número de funções (k) e do volume de dados.

Em sala de aula vimos que algoritmos como **SHA-256** e **Keccak-256** que são utilizados em sistemas de blockchain e segurança. Nesta atividade avaliamos o custo computacional de empregá-las como base de um Filtro de Bloom e discutimos quando esse custo é ou não justificado.

Implementação das Estratégias de Hash

Implemente uma estratégia utilizando **double hashing** para gerar k posições a partir de duas chamadas base. A ideia é obter k índices distintos a partir de apenas dois hashes, reduzindo o número de chamadas criptográficas. Consulte o código-base para ver como essa técnica está implementada nas estratégias `hash_md5_sha1` e `hash_xxhash` e utilize-as como referência para a sua implementação.

Cada grupo deverá implementar a estratégia de hash que lhe foi atribuída conforme a tabela disponibilizada pelo professor. Todos os algoritmos listados estão disponíveis na-

tivamente na biblioteca `hashlib` do Python, independentemente do algoritmo atribuído, a estrutura do double hashing segue o mesmo padrão:

```
h1 = hashlib.new('nome_do_algoritmo', item + b'0').hexdigest()
h2 = hashlib.new('nome_do_algoritmo', item + b'1').hexdigest()

for i in range(k):
    pos = (int(h1, 16) + i * int(h2, 16)) % m
```

Após implementar e executar os experimentos com o algoritmo atribuído, **compare os resultados obtidos com os das estratégias já implementadas no notebook** (`hash_md5_sha1` e `hash_xxhash`), discutindo diferenças de desempenho e qualidade de distribuição.

Atenção

Caso o algoritmo atribuído ao seu grupo não apareça em `hashlib.algorithms_available`, instale a biblioteca correspondente via `pip` e consulte o professor.

Configuração do Experimento

- **Tamanho do filtro:** $m = 2^{26}$ bits
- **Número de funções hash:**
 - $k_1 = 4$, valor ótimo para $p \approx 5\%$ com $n = 10^7$
 - $k_2 = 8$, dobro do valor ótimo
- **Volume de inserções:** 100.000, 1.000.000 e 10.000.000 elementos.
- **Entrada:** inteiros sequenciais convertidos para string (`str(i)` para i de 1 a n).

Atenção

Espera-se que cada combinação (estratégia $\times k$) leve entre 1 e 3 minutos nesse volume. Então prepara-se...

Métricas

Para cada combinação (estratégia $\times k \times$ volume), meça e registre:

- a) **Tempo total de inserção** de todos os n elementos (em segundos).
- b) **Tempo médio por inserção** = tempo total $/n$.
- c) **Tempo de consulta:** após todas as inserções, realize 10.000 consultas a elementos *não inseridos* e meça o tempo total.

Dica

Use `time.perf_counter()` para medições.

Visualizações sugeridas

- a) **Gráfico de barras agrupadas:** tempo total de inserção por estratégia, agrupado por volume de elementos (uma série para k_1 , outra para k_2).
- b) **Gráfico de linhas:** tempo médio por inserção vs. volume de elementos (uma linha por estratégia; separe k_1 e k_2 em subplots ou com estilos de linha distintos).

Questões

Responda cada questão em uma célula Markdown separada, logo após o gráfico ou resultado correspondente.

Q1. Comparação entre estratégias: qual função hash apresentou o menor tempo médio por inserção? O resultado era esperado, dado o que foi discutido em aula sobre a complexidade computacional de SHA-256, BLAKE2b e Keccak-256?

Q2. Impacto de k_1 vs. k_2 : ao dobrar o número de funções hash, o tempo de inserção dobrou proporcionalmente? O comportamento foi uniforme entre as três estratégias?

Q3. Escalabilidade: o tempo médio por inserção se manteve constante à medida que o volume cresceu de 10K para 1M? Se houve variação, qual é a hipótese mais provável para explicá-la?

Q4. Trade-off para sistemas reais: suponha que você precisa proteger um sistema de blockchain com um Filtro de Bloom contendo 1 bilhão de itens. Qual estratégia você escolheria e por quê? Considere velocidade de inserção, qualidade de distribuição e o requisito de segurança criptográfica do contexto blockchain.

Q5. Taxa de falsos positivos observada: após inserir 10.000.000 de elementos com $m = 2^{26}$, realize 10.000 consultas a elementos não inseridos e reporte a taxa de falsos positivos empírica para $k_1 = 4$ e $k_2 = 8$. Os valores obtidos eram esperados? Justifique com base nos parâmetros do filtro.

Q6. Extra: observe a implementação da estratégia `hash_xxhash` no notebook e compare-a com o double hashing utilizado nas demais estratégias. Identifique a diferença na forma como os k índices são gerados e discuta: essa abordagem produz índices com distribuição equivalente ao double hashing clássico? Há alguma vantagem ou desvantagem em utilizá-la?

Como responder este laboratório

Este laboratório é entregue como um **Jupyter Notebook** (.ipynb). Cada atividade combina células de código Python com células de texto em **Markdown**, onde você vai escrever suas respostas, análises e justificativas.

Siga as instruções abaixo rigorosamente:

1. **Células de código:** implemente o que é pedido, execute a célula e certifique-se de que a saída (gráficos, tabelas, valores) está visível no notebook antes de entregar.
2. **Células de resposta em Markdown:** para cada questão de reflexão ou análise, insira uma nova célula do tipo *Markdown* logo abaixo da célula de código corres-

pondente. Para criar uma célula Markdown no Jupyter, clique no menu suspenso que aparece como “Code” na barra de ferramentas e selecione “Markdown”. Escreva sua resposta em texto corrido, com argumentação clara. Não é suficiente apenas mostrar números — explique o raciocínio.

3. **Equações no Markdown:** quando precisar escrever fórmulas matemáticas nas suas respostas, use a notação $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$. Use $\$...\$$ para fórmulas *inline* (no meio do texto) e $\\\$...\$$ para fórmulas em bloco (centralizadas, em linha própria). O Jupyter renderiza essas fórmulas automaticamente quando a célula é do tipo Markdown. Se não estiver familiarizado com essa notação, consulte:

<https://www.markdownguide.org/extended-syntax/#latex>

Exemplo *inline*: escreva A taxa de falsos positivos é $p = 0{,}01$

Exemplo em bloco:

```
$$  
k = \frac{m}{n} \ln 2  
$$
```

4. **Exportação para PDF:** ao finalizar, exporte o notebook pelo menu File → Save and Export Notebook As → PDF. Abra o arquivo gerado e **verifique se todas as células, gráficos e equações estão legíveis** antes de submeter. Caso o seu ambiente não suporte exportação direta, utilize File → Print → Salvar como PDF pelo navegador.

Atenção

Entregas com células não executadas, gráficos ausentes, equações não renderizadas ou PDF ilegível não serão aceitas. Execute o notebook do início ao fim com Kernel → Restart & Run All antes de exportar.

Outras Estruturas de Dados Probabilísticas

Filtros de Bloom fazem parte de uma família de estruturas de dados probabilísticas que utilizam funções hash para oferecer respostas aproximadas com custo de memória muito inferior ao das estruturas exatas. A seguir são apresentadas quatro estruturas:

Count-Min Sketch. Proposta por Cormode e Muthukrishnan (2005), essa estrutura **estima a frequência de ocorrência de elementos em um fluxo de dados** sem armazená-los explicitamente. Utiliza uma matriz de contadores indexada por múltiplas funções hash independentes; a estimativa de frequência de um elemento é o mínimo entre os contadores correspondentes. É amplamente utilizada em análise de tráfego de rede, detecção de elementos frequentes e sistemas de recomendação em tempo real. Referência: <https://dl.acm.org/doi/10.1145/1073814.1073816>

HyperLogLog. Introduzida por Flajolet et al. (2007), estima a **cardinalidade de um conjunto** - isto é, o número de elementos distintos - usando quantidade de memória logarítmica em relação ao número de elementos. Baseia-se na observação do padrão de bits iniciais dos valores hash para inferir estatisticamente o tamanho do conjunto. É

utilizada em bancos de dados como Redis e Google BigQuery para contagens distintas em escala de bilhões de elementos com erro típico inferior a 2%. Referência: <https://algo.inria.fr/flajolet/Publications/FlFuGaMe07.pdf>

Cuckoo Filter. Proposto por Fan et al. (2014) como alternativa ao Filtro de Bloom, o Cuckoo Filter armazena impressões digitais (*fingerprints*) dos elementos em uma tabela hash de cuco. Oferece as mesmas garantias de consulta probabilística do Filtro de Bloom, mas com duas vantagens importantes: suporte a **deleção**. Referência: <https://dl.acm.org/doi/10.1145/2674005.2674994>

Quotient Filter. Apresentado por Bender et al. (2012), armazena quocientes dos valores hash em uma tabela compacta, o que resulta em melhor localidade de cache em comparação ao Filtro de Bloom. Suporta deleção, redimensionação. Referência: <https://dl.acm.org/doi/10.1145/1993636.1993712>